

Grupo ARCOS

uc3m | Universidad **Carlos III** de Madrid

Tema 3: Fundamentos de la programación en ensamblador (IV) **Estructura de Computadores**

Grado en Ingeniería Informática
Grado en Matemática aplicada y Computación
Doble Grado en Ingeniería Informática y Administración de Empresas



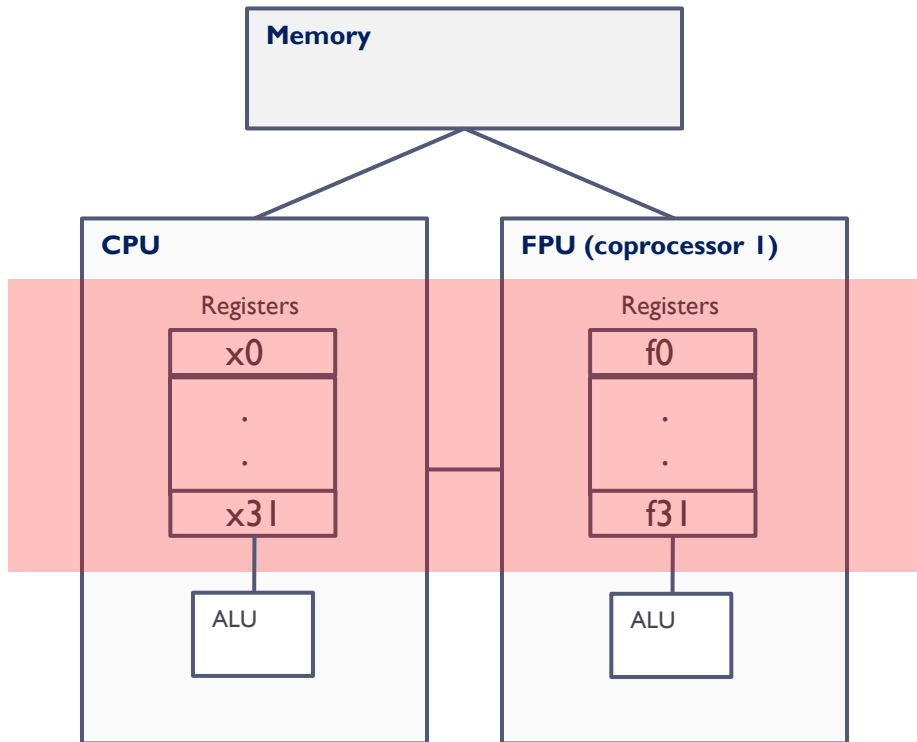
Contenidos

1. Fundamentos básicos de la programación en ensamblador
2. Ensamblador del RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. **Coma flotante en ensamblador**
 - ▶ Registros
 - ▶ Transferencia de datos
 - ▶ Operaciones aritméticas
 - ▶ Detectar y comparar
 - ▶ Memoria
5. Llamadas a procedimientos y uso de la pila

Contenidos

1. Fundamentos básicos de la programación en ensamblador
2. Ensamblador del RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. Coma flotante en ensamblador
 - ▶ Registros
 - ▶ Transferencia de datos
 - ▶ Operaciones aritméticas
 - ▶ Detectar y comparar
 - ▶ Memoria
5. Llamadas a procedimientos y uso de la pila

RISC-V 32



▶ Registros

- ▶ Dentro del procesador
- ▶ Instrucciones trabajan con valores en registros
- ▶ 32 registros de 32 bits

▶ Memoria

- ▶ Fuera del procesador
- ▶ Intercambio de datos a/desde registros
- ▶ Más capacidad pero más tiempo de acceso que los registros
- ▶ Direcciones de 32 bits
 - ▶ 4 GiB direccionable

Banco de registros (enteros)

resumen

Nombre ABI	Número	Uso
zero	x0	Constante 0
ra	x1	Dirección de retorno (rutinas)
sp	x2	Puntero a pila
gp	x3	Puntero al área global
tp	x4	Puntero al hilo
t0...t2	x5-x7	Temporal (<u>NO</u> se conserva entre llamadas)
s0 / fp	x8	Temporal (se conserva entre llamadas) / Puntero a marco de pila
s1	x9	Temporal (se conserva entre llamadas)
a0...a1	x10...11	Argumento de entrada para rutinas/valores de retorno
a2...a7	12...x17	Argumento de entrada para rutinas
s2... s11	x18...x27	Temporal (se conserva entre llamadas)
t3...t6	x28...x31	Temporal (<u>NO</u> se conserva entre llamadas)

- ▶ Hay 32 registros
 - ▶ 4 bytes de tamaño (una palabra)
 - ▶ Doble nombrado:
 - ▶ Lógico: nombre ABI (*Application Binary Interface*)
 - ▶ Numérico: con **x** al principio
- ▶ Convenio de uso
 - ▶ Reservados
 - ▶ Argumentos
 - ▶ Resultados
 - ▶ Temporales
 - ▶ Punteros

Banco de registros (coma flotante)

Nombre ABI	Nombre	Uso
ft0...ft7	f0 ... f7	Temporales (como los t...)
fs0...fs1	f8 ... f9	Se guardan (como los s...)
fa0...fa1	f10 ... f11	Argumentos/retorno (como los a...)
fa2...fa7	f12 ... f17	Argumentos (como los a...)
fs2...fs11	f18 ... f27	Se guardan (como los s...)
ft8...ft11	f28 ... f31	Temporales (como los t...)

- ▶ Hay 32 registros
- ▶ El registro ft0 no tiene su valor a 0
- ▶ En la extensión de simple precisión los registros son de 32 bits (4 bytes)
- ▶ En la extensión de doble precisión los registros son de 64 bits (8 bytes) y pueden almacenar:
 - ▶ Valores de simple precisión en los 32 bits inferiores del registro
 - ▶ Valores de do le precisión en los 64 bits del registro

Instrucciones de copiar

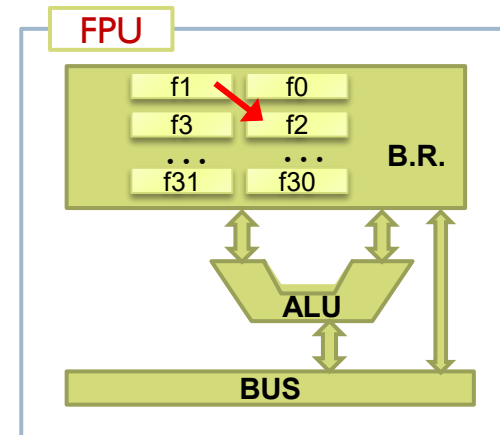
registros simple precisión <-> registros simple precisión

► Copiar datos:

- Entre registros de la **FPU**
- Entre registros de **CPU** y **FPU**

► Ejemplos:

- Copia de registro a registro (.s .d):
`fmv.s f2, f1` # rd = rs

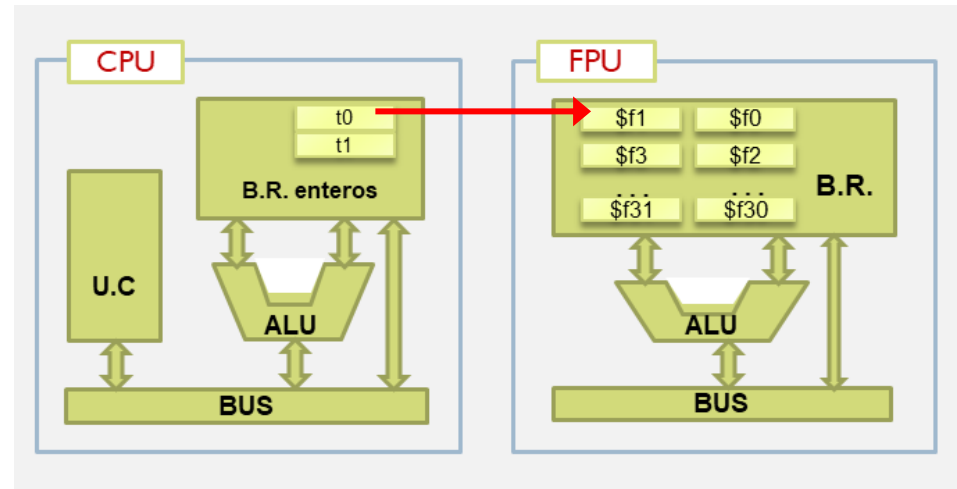


Operaciones de copia

registros enteros $\leftrightarrow$ registros coma flotante

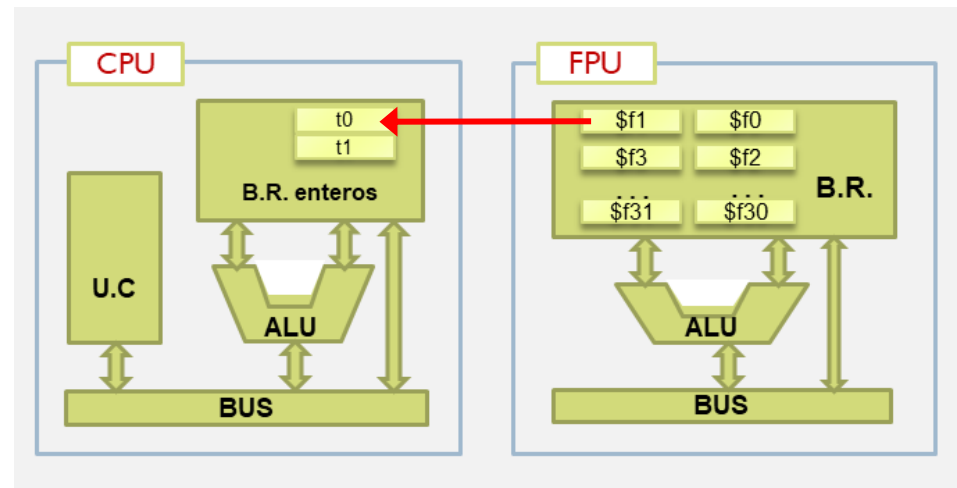
`fmv.w.x rds rsw`

- Copia desde reg. entero *rs* a reg. flotante *rd* (*single precisión*)



`fmv.x.w rdw rss`

- Copia desde reg. flotante *rs* (*single precisión*) a reg. entero *rd*



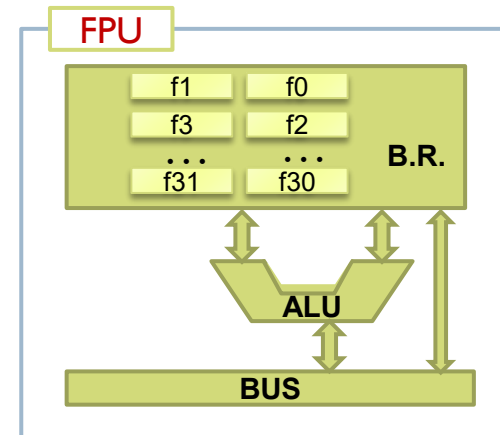
Instrucciones de copiar

registros simple precisión <-> registros simple precisión

- ▶ No existe fli.s y fli.d (*):
 - ▶ La extensión Zfa ofrece fli.s y fli.d ...
 - ▶ ... pero estas instrucciones usan un índice de 5 bits para cargar uno de los 32 valores constantes más usados en el software científico y matemático (1.0, 0.5, min, inf, etc.)

- ▶ Ejemplo:

▶ ~~Carga inmediata~~
fli.s f0, 5.0



Ejemplo

```
float PI = 3,1415;
```

```
.text
```

```
main:
```

```
# * No existe
```

```
# fli.s ft0 3,1415
```

```
# * 0x40490E56 es la
```

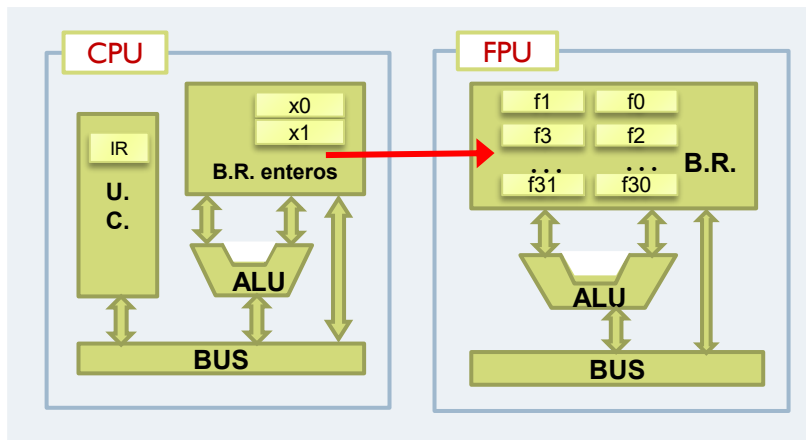
```
# representación IEEE754
```

```
# en hexadecimal de 3,1415
```

```
li      t0, 0x40490E56
```

```
fmv.w.x ft0, t0
```

```
jr ra
```



Operaciones de conversión (1 / 3)

doble precisión $\leftrightarrow$ simple precisión

► `fcvt.s.d rd, rs1`

- Convierte de doble precisión (valor en registro flotante rs) a simple precisión (registro flotante rd).

► `fcvt.d.s rd, rs`

- Convierte de simple precisión (valor en registro flotante rs) a doble precisión (registro flotante rd).

Operaciones de conversión (2/3)

entero <-> simple precisión

► `fcvt.w.s rd, rs`

- Convierte de simple precisión (valor en registro flotante rs) a entero de 32 bits **con** signo (registro entero rd).

► `fcvt.wu.s rd, rs`

- Convierte de simple precisión (valor en registro flotante rs) a entero de 32 bits **sin** signo (registro entero rd).

► `fcvt.s.w rd, rs`

- Convierte de entero de 32 bits **con** signo (valor en registro entero rs) a simple precisión (registro flotante rd).

► `fcvt.s.wu rd, rs`

- Convierte de entero de 32 bits **sin** signo (valor en registro entero rs) a simple precisión (registro flotante rd).

Operaciones de conversión (3/3)

entero <-> doble precisión

▶ `fcvt.w.d rd, rs`

- ▶ Convierte de doble precisión (valor en registro flotante rs) a entero de 32 bits **con** signo (registro entero rd).

▶ `fcvt.wu.d rd, rs`

- ▶ Convierte de doble precisión (valor en registro flotante rs) a entero de 32 bits **sin** signo (registro entero rd).

▶ `fcvt.d.w rd, rs`

- ▶ Convierte de entero de 32 bits **con** signo (valor en registro entero rs) a doble precisión (registro flotante rd).

▶ `fcvt.d.wu rd, rs1`

- ▶ Convierte de entero de 32 bits **sin** signo (valor en registro entero rs) a doble precisión (registro flotante rd).

Ejemplo

```
float TRES = 3,0;
```

```
.text
```

```
main:
```

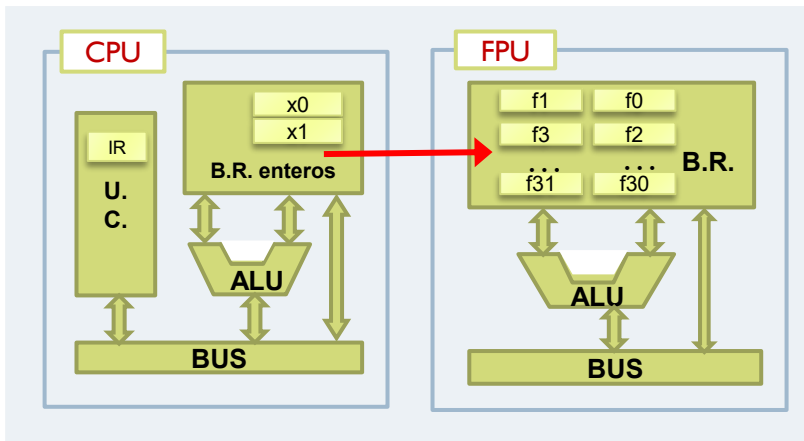
```
# * No existe
```

```
# fli.s ft0 3,1415
```

```
li      t0, 3
```

```
fcvt.s.w ft0, t0
```

```
jr ra
```

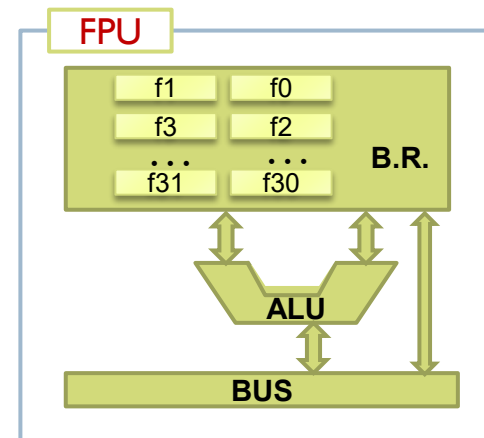


Contenidos

1. Fundamentos básicos de la programación en ensamblador
2. Ensamblador del RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. Coma flotante en ensamblador
 - ▶ Registros
 - ▶ Transferencia de datos
 - ▶ Operaciones aritméticas
 - ▶ Detectar y comparar
 - ▶ Memoria
5. Llamadas a procedimientos y uso de la pila

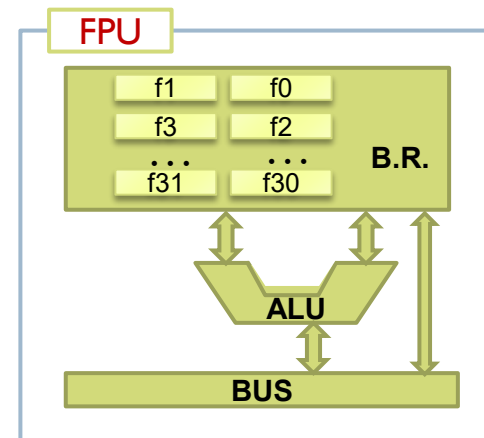
Operaciones aritméticas comunes (1/2)

- ▶ De simple (.s) y doble (.d) precisión:
 - ▶ `fadd.s rd rs1 rs2` $\# \text{rd} = \text{rs1} + \text{rs2}$
 - ▶ `fsub.s rd rs1 rs2` $\# \text{rd} = \text{rs1} - \text{rs2}$
 - ▶ `fmul.s rd rs1 rs2` $\# \text{rd} = \text{rs1} * \text{rs2}$
 - ▶ `fdiv.s rd rs1 rs2` $\# \text{rd} = \text{rs1} / \text{rs2}$
 - ▶ `fabs.s rd rs` $\# \text{rd} = |\text{rs}|$
 - ▶ `fneg.s rd rs` $\# \text{rd} = -\text{rs}$



Operaciones aritméticas comunes (2/2)

- ▶ De simple (.s) y doble (.d) precisión:
 - ▶ `fmin.s rd rs1 rs2` # $rd = \min(rs1, rs2)$
 - ▶ `fmax.s rd rs1 rs2` # $rd = \max(rs1, rs2)$
 - ▶ `fsqrt.s rd rs1` # $rd = \sqrt{rs1}$
 - ▶ `fmadd.s rd rs1 rs2 rs3` # $rd = rs1 \times rs2 + rs3$
 - ▶ `fmsub.s rd rs1 rs2 rs3` # $rd = rs1 \times rs2 - rs3$



Ejemplo

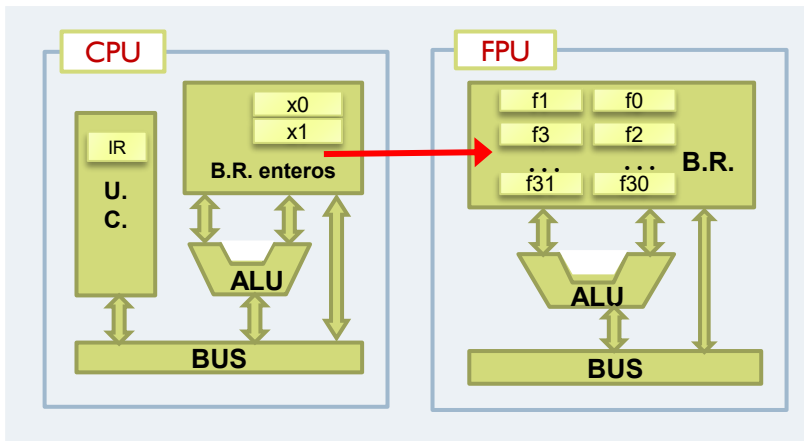
```
float PI    = 3,1415;
int  radio = 4;
float length;

length = PI * radio;
```

```
.text
main:
```

```
li      t0, 0x40490E56
fmv.w.x ft0, t0    # ft0 ← t0
li      t1,  4     # 4 en Ca2
fcvt.s.w ft1, t1    # 4 en ieee754
fmul.s  ft0, ft0, ft1

jr ra
```



Contenidos

1. Fundamentos básicos de la programación en ensamblador
2. Ensamblador del RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. Coma flotante en ensamblador
 - ▶ Registros
 - ▶ Transferencia de datos
 - ▶ Operaciones aritméticas
 - ▶ Detectar y comparar
 - ▶ Memoria
5. Llamadas a procedimientos y uso de la pila

Clasificación de números en coma flotante

- ▶ `fclass.s rd, rs1` (simple precisión)
- ▶ `fclass.d rd, rs1` (doble precisión)
- ▶ Escribe en `rd` el tipo de número de coma flotante del registro `rs1`:

Bit en rd	Significado
0	-Inf
1	Número normalizado negativo
2	Número no normalizado negativo
3	-0
4	+0
5	Numero no normalizado positivo
6	Número normalizado positivo
7	+Inf
8	NaN (no silencioso)
9	NaN (silencioso)

Estándar IEEE 754 de precisión simple

resumen

Exponente	Mantisa	Valor especial
0 (0000 0000)	0	$(-1)^s * 0.\text{mantisa} * 2^{-126}$ +/- 0 (según signo)
0 (0000 0000)	No cero	Número NO normalizado
255 (1111 1111)	No cero	NaN (0/0,...)
255 (1111 1111)	0	+/-infinito (según signo)
1-254	Cualquiera	Número normalizado (no especial) $(-1)^s * 1.\text{mantisa} * 2^{\text{exponente}-127}$

Ejemplo

```
float PI = 3,0;

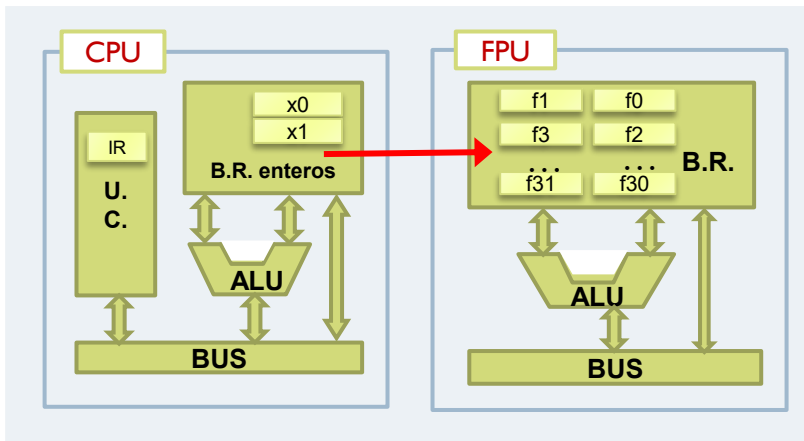
print(typeof(PI));
```

```
.text
main:
```

```
# ft0 ← 3,0
li      t0, 3
fcvt.s.w ft0, t0
fclass.s t1, ft0
```

```
# print(typeof(ft0))
li a7, 1
mv a0, t1
ecall

jr ra
```



Instrucciones de comparación (registros en coma flotante)

► Simple precisión

- `feq.s rd, rs1, rs2` if (rs1 == rs2) rd=1; else rd=0
- `fle.s rd, rs1, rs2` if (rs1 <= rs2) rd=1; else rd=0
- `flt.s rd, rs1, rs2` if (rs1 < rs2) rd=1; else rd=0

► Doble precisión:

- `feq.d rd, rs1, rs2` if (rs1 == rs2) rd=1; else rd=0
- `fle.d rd, rs1, rs2` if (rs1 <= rs2) rd=1; else rd=0
- `flt.d rd, rs1, rs2` if (rs1 < rs2) rd=1; else rd=0

Bifurcaciones con coma flotante

```
x = 3.0;  
y = 2.0;
```

```
if (x < y)  
{  
    . . .  
}
```

```
.text
```

```
main:
```

```
li      t0, 3  
fcvt.s.w ft0, t0
```

```
li      t1, 2  
fcvt.s.w ft1, t1
```

```
flt.s t0, ft1, ft2  
bne t0, x0, fin_if1  
# . . .  
fin_if1:
```

```
jr ra
```

Contenidos

1. Fundamentos básicos de la programación en ensamblador
2. Ensamblador del RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. Coma flotante en ensamblador
 - ▶ Registros
 - ▶ Transferencia de datos
 - ▶ Operaciones aritméticas
 - ▶ Detectar y comparar
 - ▶ Memoria
5. Llamadas a procedimientos y uso de la pila

Representación de tipos básicos (1/3)

```
// booleanos
bool_t b1 ;
bool_t b2 = false ;
```

```
// caracteres
char c1 ;
char c2 = 'x' ;
```

```
// enteros
int res1 ;
int op1 = -10 ;
```

```
// coma flotante
float f0 ;
float f1 = 1.2 ;
double d2 = 3.0e10 ;
```

```
.data

# boolean
b1:    .zero    1          # 1 bytes
b2:    .byte    0          # 1 byte

# character
c1:    .byte    'x'        # 1 bytes
c2:    .byte    'x'        # 1 bytes

# integers
res1:  .zero    4          # 4 bytes
op1:   .word    -10        # 4 bytes

# floating point
f0:    .float    1.2        # 4 bytes
f1:    .float    1.2        # 4 bytes
d2:    .double   3.0e10     # 8 bytes
```

Definición de datos estáticos

(en tiempo de compilación)

etiqueta (dirección)

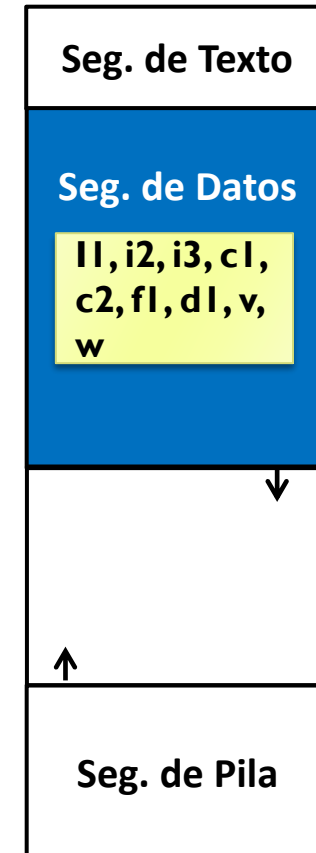
tipo de dato (directiva)

valor

```
.data
cadena .string "Hola mundo\n"
i1: .word 10      # int    i1=10
i2: .word -5      # int    i2=-5
i3: .half 300     # short  i3=300
c1: .byte 100     # char   c1=100
c2: .byte 'a'     # char   c2='a'
f1: .float 1.3e-4 # float  f1=1.3e-4
d1: .double .001  # double d1=0.001

# int v[3]={0,-1,0xffffffff}; int w[100];
v: .word 0, -1, 0xffffffff
w: .zero 400
```

0x00



Operaciones de memoria: coma flotante

Acceso a memoria (coma flotante)		
<code>flw rs, 10(rs1)</code>	<code>rs = Memoria[rs1+10]</code>	Carga el valor simple precisión almacenado en la dirección (rs1+10) en el registro de coma flotante rd.
<code>fsw rs, 10(rs1)</code>	<code>Memoria[rs1+10] = rs</code>	Almacena el valor simple precisión del registro rs en la dirección (rd1+10).
<code>fld rd, 10(rs1)</code>	<code>rd = Memoria[rs1+10]</code>	Carga el valor doble precisión almacenado en la dirección (rs1+10) en el registro rd.
<code>fsd rd, 10(rs1)</code>	<code>Memoria[rs1+10] = rd</code>	Almacena el valor doble precisión del registro rs en la dirección (rd1+10).

Tipo de datos básicos

float

```
float  resultado ;  
float  op1 = 100 ;  
float  op2 = 2.5
```

...

```
main ()
```

```
{  
    resultado = op1 + op2 ;  
    ...  
}
```

.data

```
.align 2  
resultado: .zero 4 # 4 bytes  
op1:      .float 100  
op2:      .float 2.5  
# ...
```

.text

```
main:  la      t0,  op1  
       flw     ft0, 0(t0)  
       la      t0,  op2  
       flw     ft1, 0(t0)  
       fadd.s  ft3, ft1, ft2  
       la      t0,  resultado  
       fsw     ft3, 0(t0)  
# ...
```

```
jr ra
```

Tipo de datos básicos

double

```
double resultado ;  
double op1 = 100 ;  
double op2 = 2.5
```

```
...
```

```
main ()
```

```
{
```

```
    resultado = op1 + op2 ;
```

```
    ...
```

```
}
```

```
.data
```

```
    .align 2
```

```
    resultado: .zero 8    # 8 bytes
```

```
    op1:      .double 100
```

```
    op2:      .double 2.5
```

```
    # ...
```

```
.text
```

```
main:  la      t0,  op1
```

```
        fld    ft0, 0(t0)
```

```
        la     t0,  op2
```

```
        fld    ft1, 0(t0)
```

```
        fadd.s ft3, ft1, ft2
```

```
        la     t0,  resultado
```

```
        fsd    ft3, 0(t0)
```

```
        # ...
```

```
        jr ra
```

Ejemplo. Calcular

$$\sum_{i=1}^N \frac{1}{2^i}$$

```
.data

N:          .word  10          # Límite superior
one_f:      .float 1.0         # 1.0 en decimal
two_f:      .float 2.0         # 2.0 en decimal

.text

main:
    la      t0, N
    lw      t0, 0(t0)          # t0 = N = 10

    la      t1, one_f
    flw     ft0, 0(t1)         # ft0 = 1.0 (numerador)

    la      t1, two_f
    flw     ft1, 0(t1)         # ft1 = 2.0 (base de la potencia)

    fmv.w.x fa0, zero          # acumular el resultado

    # Inicializar variables del bucle
    li      t1, 1              # t1 = i = 1
    fmv.s   ft2, ft1           # ft2 = 2.0 (representará 2^i, empezando en 2^1)

    # continua...
```

Ejemplo. Calcular

$$\sum_{i=1}^N \frac{1}{2^i}$$

loop:

```
bgt t1, t0, end_loop
```

```
# Calcular el término en curso: ft3 = 1.0 / (2^i)
```

```
fdiv.s ft3, ft0, ft2 # ft3 = ft0 / ft2
```

```
# Sumar al acumulador: fa0 = fa0 + ft3
```

```
fadd.s fa0, fa0, ft3
```

```
# Actualizar la siguiente iteración
```

```
fmul.s ft2, ft2, ft1 # multiplica por 2 para la siguiente potencia (2^(i+1))
```

```
addi t1, t1, 1      # i = i + 1
```

```
j loop
```

end_loop:

```
# imprime el resultado
```

```
li a7, 2
```

```
ecall
```

```
li a7, 10    # syscall 10 = exit
```

```
ecall
```

Ejercicio

- ▶ Escriba un programa que:
 - ▶ Cargue el valor -3.141516 en el registro f0
 - ▶ Permita obtener el valor del exponente y de la mantisa almacenada en el registro f0 (en formato IEEE 754)
 - ▶ Imprima el signo
 - ▶ Imprima el exponente
 - ▶ Imprima la mantisa

Ejercicio (Solución)

```
.data
    valor:    .float -3.141516

.text
main:
    la    t0, valor
    flw   ft0, 0(t0)

    #se imprime
    fmv.s fa0, ft0
    li    a7, 2
    ecall

    li a0, '\n' # salto de línea
    li a7, 11
    ecall

    # se copia al procesador
    fmv.x.w    t0, ft0
```

```
li    t1, 0x80000000 # signo
and a0, t0, t1
srli a0, a0, 31
li    a7, 1
ecall

li a0, '\n' # salto de línea
li    a7, 11
ecall

li    t1, 0x7F800000 # exponente
and a0, t0, t1
srli a0, a0, 23
li    a7, 1
ecall

li a0, '\n' # salto de línea
li    a7, 11
ecall

li    t1, 0x007FFFFF # mantisa
and a0, t0, t1
li    a7, 1
ecall

jr ra
```

Grupo ARCOS

uc3m | Universidad **Carlos III** de Madrid

Tema 3: Fundamentos de la programación en ensamblador (IV) **Estructura de Computadores**

Grado en Ingeniería Informática
Grado en Matemática aplicada y Computación
Doble Grado en Ingeniería Informática y Administración de Empresas

